

CENG 201 – Task 3 Report

Topic: Discharge Stack Implementation using Stack

In this task, a stack data structure was implemented to manage discharged patients in a hospital management system. The main purpose of this task is to demonstrate the use of the Last-In-First-Out (LIFO) principle and to evaluate how a stack can be effectively used to store discharge records.

Each discharged patient is represented by a discharge record that contains the patient ID and the discharge time. The most recently discharged patient is placed on the top of the stack, allowing quick access to the latest discharge information.

Time Complexity Analysis:

In this task, the stack was implemented using a singly linked list. The push operation adds a new discharge record to the top of the stack by updating the head reference. Since no traversal is required, the time complexity of this operation is $O(1)$.

The pop operation removes the most recent discharge record from the stack. This operation also only involves accessing and updating the head node of the linked list, resulting in a constant time complexity of $O(1)$.

Similarly, the peek operation returns the top element of the stack without removing it. As it directly accesses the head node, its time complexity is also $O(1)$.

Stack vs Queue Comparison:

A stack is suitable for managing discharge records because recently discharged patients are more likely to be accessed or reviewed. The LIFO structure allows efficient access to the most recent discharge information.

If a queue were used instead, discharge records would be processed in a First-In-First-Out (FIFO) order. Although queue operations also have a time complexity of $O(1)$,

accessing the most recent discharge record would not be as efficient. Therefore, a stack is a more appropriate data structure for this hospital management scenario.